Tarea 1: Ejercicio: Verificación vs. Validación con apoyo de IA
Pruebas de Software
02 de Abril 2026

URL REPO: https://github.com/GasparNavarro/Tarea-1-Pruebas-De-Software/

Integrantes:
Diego Villouta 2773019-1
Gaspar Navarro 202173003-6
Benjamín Paulsen 202173017-6

ÍNDICE

1. Introducción....3
2. Verificación vs Validación....3
3. Validación del requerimiento mediante su análisis....4
4. Requerimiento mejorado....6
5. Reglas del negocio....7
6. Alcance y exclusiones....7
7. Verificación....8
8. Diseño e implementación general....9
9. Estrategia de pruebas....9
10. Casos de prueba....10
11. Pruebas que validan nuestros supuestos....10
12. Uso de IA en el proyecto....11
13. Mejoras y trabajo futuro....11
14. Reflexión final....13
15. Conclusiones....13
16. Evidencias del trabajo....14

1. Introducción

Este trabajo tiene como objetivo principal aplicar conceptos de validación y verificación mediante el análisis de un requerimiento de software incompleto relacionado a la gestión de reservas de atención técnica de electrodomésticos. Además de mejorarlo, se implementará una solución mediante una aplicación web simple, y se aplicarán conceptos de testing para verificar que el sistema cumple con el requerimiento mejorado. También se dará detalle del uso de IA en el proceso.

2. Verificación vs Validación

Se distingue la Calidad asegurada (QA) enfocada en el proceso y el Control de Calidad (QC) enfocada en el producto.

- **Verificación:** se centra en comprobar que el software cumple con las especificaciones técnicas y de diseño que definimos.
    - **Actividad 1:** Ejecución de pruebas unitarias sobre modelos de Django para asegurar la regla de superposición de reservas tal que funcione a nivel de BD.
    - **Actividad 2:** Análisis estático de código con flake8 o pylint en nuestro pipeline local para asegurar codificación en Python.
    - **Actividad 3:** Revisión de código por pares (PR Github) antes de fusionar ramas de características hacia develop.
- **Validación:** centrada en asegurar que el software resuelva la necesidad real del negocio (tomar reservas telefono/whatsapp).
    - **Actividad 1:** Pruebas de usabilidad simuladas, tal que uno de nosotros dicta datos por teléfono simulando ser cliente y el otro intenta ingresar la reserva en el sistema para medir tiempos de fricción.
    - **Actividad 2:** Realizar el flujo completo correctamente, tal de crear cliente, asignar técnico, cancelar, validando que el estado final sea coherente con operación de la tienda.
    - **Actividad 3:** Revisión de interfaz, tal de verificar en pantalla de futuras reservas, se vea la información necesaria (teléfono cliente, falla) de una sola vez sin clickear demás.

3. Validación del requerimiento mediante su análisis

El requerimiento inicial es el siguiente:

> "Una pequeña empresa necesita una aplicación web para gestionar reservas de atención técnica de electrodomésticos.
>
> El sistema debe permitir, al menos:
>
> - Registrar clientes
> - Registrar técnicos
> - Agendar una reserva de atención técnica
> - Cancelar una reserva
> - Consultar las reservas futuras
>
> El sistema lo utilizará la empresa de servicio técnico, que toma los pedidos por email y Whatsapp
>
> La empresa menciona que desea una solución “simple, usable y confiable”, pero no entrega mayores detalles."

A partir de este enunciado, y para poder validar que efectivamente estamos resolviendo el problema correcto, se realizan las siguientes preguntas:

- **"Una pequeña empresa necesita una aplicación web para gestionar reservas de atención técnica de electrodomésticos":**
    - ¿Qué tan pequeña es la empresa?, (cantidad de trabajadores), esto puede afectar en la cantidad de usuarios simultáneos de la aplicación, la cantidad de datos a manejar, la infraestructura necesaria, el tipo de hosting a utilizar.
    - Empresa se encarga solamente de asignar técnicos a clientes?, cobra a los clientes por servicio de asignación?, por la reparación?, ambos?. Quizás la empresa cobra una cantidad "X" a los clientes por asignar a técnico, y luego este le cobra "Y" por la reparación?.
    - Se cobra por visita técnica?, por electrodoméstico reparado?, por hora de trabajo?. ¿Qué medio se utiliza para cobrar?, si se comunican por email/whatsapp puede que los pagos sean por transferencia electrónica.
    - Empresa mantiene contratados a técnicos, o son independientes y están registrados en el sistema a modo de bolsa de trabajo?. Si son independientes, ¿se les cobra por registrarse en el sistema como técnico disponible?
    - Cómo se entera el técnico de que tiene una nueva reserva?, se le envía un correo electrónico, un mensaje de whatsapp?, de qué manera se le notifica?
    - Sistema opera de manera 100% local, o se puede acceder desde cualquier lugar?. Dado que es una aplicación web, se asume que se puede acceder desde cualquier lugar con conexión a internet. Sobre todo si agrega que toma los pedidos por email y whatsapp, esto hace suponer que se tiene acceso a internet.

    - Se solicita atención según una categoría de electrodoméstico, o según un modelo específico?. Los electrodomésticos o categorías de electrodomésticos se tendrían previamente registradas en el sistema, o se registraron al momento de la reserva?, ambos?
- **"El sistema lo utilizará la empresa de servicio técnico":** Da a entender que no será de uso público. Clientes no podrán acceder al sistema. Solo gente de la empresa.
- **"Registrar clientes":**
    - Qué información se debe registrar de un cliente?, puede ser editada posteriormente?
    - Se pueden eliminar clientes o sólo se dan de baja?, qué pasa con sus reservas en este caso?
- **"Registrar técnicos":**
    - ¿Qué información se debe registrar de un técnico?, puede ser editada posteriormente?
    - Se pueden eliminar técnicos o sólo se dan de baja?, qué pasa con sus reservas asignadas en este caso?
- **"Agendar una reserva de atención técnica":**
    - ¿Qué información se debe registrar de una reserva?:
        - Fecha y hora. Debe ser mayor a la fecha y hora actual. Por lo menos 1 hora después. Igualmente, ¿Qué pasa si el técnico está muy lejos de un cliente con respecto al cliente actual, en bloques consecutivos? Habría que manejar la logística de ubicaciones de los clientes, y técnicos, al momento de realizar una reserva.
        - Id del técnico. Debe estar disponible en la fecha y hora de la reserva. No puede tener otra reserva en el mismo horario.
        - Id del cliente. Debe estar registrado en el sistema antes de intentar crear la reserva. Podría ser el RUT.
        - Categoría de electrodoméstico a reparar. Debe existir en el sistema. El técnico asignado debe tener especialidad en esa categoría.
        - Descripción del problema.
        - Estado. Puede ser "pendiente", "asignada", "completada", "cancelada". Este dato se puede
    - Podría un técnico tener más de una reserva en el mismo día?
- **"Cancelar una reserva":**
    - Quién puede cancelar una reserva?
    - Qué reservas pueden ser canceladas?
    - Cancelar implica cambiar su estado a "cancelada"?
- **"Consultar las reservas futuras":**
    - Deberían ver todas las reservas. Las reservas pasadas podrían aparecer con un estado de "completada", "cancelada", o "expirada", y las futuras con un estado de "pendiente" o "asignada". Algunos de estos estados podrían ser "calculados" según ciertas condiciones. Por ejemplo, si una reserva tiene una fecha y hora menor a la fecha y hora actual, su estado pasaría a ser "expirada". El resto puede ser asignado manualmente.
- **Puede editarse una reserva?, o se crearía una nueva?**
- **"La empresa menciona que desea una solución “simple, usable y confiable”":**
    - ¿Qué significa "simple"? ¿Qué significa "usable"? ¿Qué significa "confiable"?:

        - Digamos que simple significa que cada acción que se quiera realizar, requiere de pocos pasos, (no más de 2 o 3). Que no requiera de conocimientos previos para su uso.
        - Usable significa que la interfaz sea intuitiva, fácil de usar, y que no requiera de conocimientos previos para su uso. Que funcione para la intención base de asignar reservas.
        - Confiable significa que el sistema sea estable y no falle. Que no se pierdan datos.
- **Quienes tienen acceso al sistema?, todos pueden hacer las mismas acciones?**
    - Tipos de usuario que usan la aplicación, niveles de acceso:
        - Administrador: Puede hacer todo. Puede ser técnico también.
        - Colaborador: Puede ver sus reservas y cancelar las que le pertenecen.
        - Todo administrador es un colaborador, pero no todo colaborador es un administrador.

4. Requerimiento mejorado

En base al análisis anterior, se propone el siguiente requerimiento mejorado:

- Clientes pueden ser creados, listados, eliminados, y editados.
- Clientes deben ser creados en el sistema antes de poder ser asignados a una reserva.
- Para crear un cliente se requiere la siguiente información:
    - Nombre, String de máximo 50 caracteres.
    - Teléfono, mínimo 8 dígitos, máximo 12 dígitos. Sin código país.
    - Dirección, String de máximo 255 caracteres.
- Cuando se elimina un técnico, este no se borra, queda como inactivo.
- Técnicos pueden ser creados, listados, eliminados.
- Técnicos deben ser creados en el sistema antes de poder ser asignados a una reserva.
- Para los técnicos, se establecen bloques horarios fijos de 1 hora de duración, en una jornada estandarizada desde las 09:00 hasta las 18:00 hrs.
- Para crear un técnico se requiere la siguiente información:
    - Nombre, String de máximo 50 caracteres.
    - esta_activo, activo o inactivo (BOOLEANO)
- Para crear una reserva se requiere la siguiente información:
    - Fecha y hora. Debe ser mayor a la fecha y hora actual. Por lo menos 1 hora después. Seleccionable desde widget de fecha y hora, o similar, con formato DD/MM/AAAA HH:MM. Lo ideal es que el widget no permitiese seleccionar fechas menores a la fecha actual (regla del negocio: un técnico no puede tener asignada reservas en bloques que sean consecutivos).
    - Técnico, seleccionado desde el listado de técnicos. El técnico asignado debe tener disponibilidad en la fecha, hora y bloque de la reserva.
    - Cliente, seleccionado desde el listado de clientes.Ingresar:
        - Descripción del problema, String de máximo 255 caracteres.
        - Dirección, String de máximo 255 caracteres.

    - La dirección del cliente,por defecto será la registrada por el cliente. Si el cliente necesita atención en otra dirección, se debe especificar al momento de crear la reserva.
- Las reservas deben ser estables según fecha y hora.
- Para mantener la integridad referencial y la trazabilidad contable, no se permitirán eliminaciones físicas en la base de datos (Hard Delete). Se aplicará un modelo de estados (Soft Delete): las reservas pasarán a estado "CANCELADA" y los técnicos tendrán un atributo de estado inactivo (esta_activo = False).

5. Reglas del negocio

- Reglas de negocio mínimas definidas: Toda reserva debe estar asociada obligatoriamente a un cliente registrado y a un técnico activo.
- Las reservas se gestionan en bloques horarios fijos de 1 hora (de 09:00 a 18:00 hrs).
- Un técnico no puede tener más de una reserva activa (no cancelada) en el mismo bloque horario y fecha. Tampoco puede tener una reserva asignada en un bloque consecutivo a un bloque que ya tiene asignado, por temas de movilidad de un cliente al otro, en distintas ubicaciones (tiempo de transporte).
- No es posible agendar reservas con fechas en el pasado. La cancelación de una reserva es un cambio de estado (Soft Delete), manteniendo la trazabilidad.

6. Alcance y exclusiones

Si bien el requerimiento mejorado es más preciso, aún existen aspectos que podrían ser objeto de ambigüedad. Por lo tanto, se definen los siguientes alcances, supuestos, y exclusiones:

- Se asume que el sistema lo utilizará un solo tipo de usuario, el administrador. No se implementarán tipos de usuarios, o niveles de acceso.
- Se asume que será usado en Chile. Esto afecta al formato de RUT y teléfono.
- Dirección de cliente se asume que es válida, y que está dentro de territorio chileno. No se implementará un sistema de georreferenciación.
- No se implementará un sistema de autenticación. Se asume que el sistema será utilizado por un solo usuario.
- Se asume que el sistema será utilizado por una sola empresa. No se implementará un sistema multi-empresa.
- Las reparaciones serán asociadas a categorías de electrodomésticos, no a modelos específicos. Las categorías se irán creando en el sistema a medida que se agreguen técnicos que las soporten.
- No se implementará un sistema de pago. Se asume que la empresa se encarga de gestionar los pagos con los clientes y/o técnicos. Agregar un sistema de pago en línea tipo WebPay requeriría de una integración más compleja de lo que el sistema necesita.
- Solo se podrá reparar un electrodoméstico por reserva. Restricción que se implementa para asegurar que el técnico tenga tiempo suficiente para realizar la reparación y trasladarse a la siguiente reserva.

- No se implementará un sistema de notificaciones ni a usuarios, ni a técnicos. Se asume que la empresa se encarga de gestionar las notificaciones de reservas con los clientes y/o técnicos.

7. Verificación

1. Requerimiento: Registrar clientes

- **Entrada:** En Gestión de reservas clickear el botón “+clientes”. llenar el formulario con los datos del cliente a ingresar . Click en guardar.
- **Resultado:** El cliente creado ingresa a la lista de clientes existentes.

2. Requerimiento: Registrar técnicos

- **Entrada:** En Gestión de reservas clickear el botón “+tecnicos”. llenar el formulario con los datos del técnico a ingresar . Click en guardar.
- **Resultado:** El técnico creado ingresa a la lista de clientes existentes.

3. Requerimiento: Agendar una reserva de atención técnica

- **Entrada:** En Gestión de reservas clickear el botón “+Agender Reserva”. Elegir al cliente que solicitó la reserva, debe estar ingresado previamente, elegir al técnico que cumpla con la categoría requerida por el cliente. Describir el fallo/problema que tiene el electrodoméstico, y guardar la reserva.
- **Resultado:** La reserva guardada, queda registrada en la lista de Reservas Futuras.

4. Requerimiento: Cancelar una reserva

- **Entrada:** En Gestión de Reservas, en el listado de Reservas Futuras,clickear el botón "Cancelar" de una reserva existente.
- **Resultado:** El sistema intercepta la acción y redirige correctamente a la pantalla de confirmación, evitando borrados accidentales. Al confirmar nuevamente la acción clickeando el botón “Sí, Cancelar Reserva”, esta sale del listado Reservas Futuras.

5. Requerimiento: Consultar las reservas futuras

- **Entrada:** Entrar a Gestión de Reservas - Reservas Futuras.
- **Salida:** Se ve el listado de reservas futuras.

8. Diseño e implementación general

Para la implementación de la solución se optó por utilizar Django, un framework web robusto y de alto nivel basado en Python, aprovechando el patrón de MVT (modelo-vista-plantilla) que tiene. En el backend, se implementaron vistas basadas en clases para gestionar operaciones de CRUD y la validación de las reglas de negocio más importantes (como la superposición de horarios o agendamientos en el pasado) directamente a la capa de modelos.

El frontend se diseñó una interfaz alineada con el requerimiento de una solución “simple, usable, confiable”. Para esto, se usó html y CSS con el framework de BootStrap5,tal de estandarizar el diseño web por medio de herencia de plantillas y componentes responsive.

Para almacenar datos y asegurar la persistencia de datos, se usó la base de datos relacional nativa de Django SQLite, garantizando la integridad para manejar estados lógicos de las reservas (Soft Delete) y la activación o deshabilitación de técnicos.

Como herramienta de control de versiones se utilizó Git con repositorio remoto de GitHub. Para el trabajo colaborativo y la gestión del código, se aplicó una estrategia de ramificación inspirada en Gitflow, definiendo main como rama principal y develop como rama de integración. Se implementaron políticas de protección de estas ramas, exigiendo obligatoriamente la revisión y fusión de código mediante PR para evitar la inyección de errores en la línea principal de desarrollo. Actualmente, el despliegue del sistema está configurado para su ejecución, pruebas y evaluación en un entorno local.

9. Estrategia de pruebas

Dado el tamaño del equipo (3 integrantes) y el rápido ciclo de desarrollo, se adoptó una estrategia de pruebas manuales de caja negra ejecutadas de forma cruzada entre los integrantes del equipo. El desarrollador que no implementa una funcionalidad o característica fue el encargado de probar a través de su local, verificando que se cumplen las reglas de negocio y requerimientos.

Alcance de pruebas

- **In-Scope:** vistas CRUD, validación modelos de la BD, redirecciones de interfaz, control de acceso y transiciones de estado (soft delete).
- **Out-of-Scope:** pruebas de carga/estrés, automatización y compatibilidad multi-navegador.

Diseño de casos Se diseñaron y ejecutaron más de 30 pruebas manuales, las cuales están detalladas en la planilla excel adjunta, con el desarrollador de la funcionalidad y el integrante del equipo que testeo. Para esto, tenemos 3 ciclos, uno por cada desarrollador. De esta forma, cada ciclo tiene lo que implementó cada desarrollador y luego otro integrante del equipo testeo y analizo.

- **Ciclo 1:** casos enfocados en el flujo principal de creación y cancelación de reservas, evaluación de casos bordes y casos negativos.

- **Ciclo 2:** casos adicionales para evaluar integración de la gestión del ciclo de vida de los técnicos y persistencia de clientes, asegurando que el historial de reservas no se vea afectado.
- **Ciclo 3:** casos enfocados en las propuestas y mejoras del nuevo integrante Diego, que llegó después al grupo. El foco estaba en integrar sus ideas a nuestra base, desarrollarlas y probarlas, tal de tener las reglas de negocio claras y no romper nada del Ciclo 1 y 2.

10. Casos de prueba

Se adjunta el link a la hoja de cálculo donde se encuentran todos los casos de pruebas realizados para la entrega de la tarea 1.

https://docs.google.com/spreadsheets/d/1eSTAMHM0fqXE0Xdm4dwN25U1enDkiTKU/edit?gid=2046072400#gid=2046072400

11. Pruebas que validan nuestros supuestos

A lo largo del análisis y diseño, el equipo estableció una serie de supuestos y reglas de negocio para definir el alcance y viabilidad técnica. Para esto, se diseñaron y ejecutaron pruebas específicas que tensionan estos supuestos, las cuales son:

- **Supuesto 1: Sistema de uso interno**
  - **Premisa:** clientes no interactúan con aplicación web, solo operarios administradores.
  - **Validación testing:** a través de TENSION-01, se demuestra que un usuario anónimo al entrar a la url /nueva, sistema bloquea acceso y lo redirige a pantalla de inicio de sesión, con status 302, tal que protege la ruta por medio de LoginRequiredMixin
- **Supuesto 2: Historial reservas inmutable**
  - **Premisa:** mantener integridad referencial y trazabilidad, no se eliminan registros físicos de la BD ni de reservas ni de técnicos.
  - **Validación testing:** Caso FUNC-07 verifica que al cancelar una reserva, tal que desaparece de la interfaz, pero el registro sigue existiendo en la BD. El caso BORDE-06 valida que al deshabilitar un técnico, sus reservas pasadas se mantienen intactas y el sistema no arroja errores de integridad.
- **Supuesto 3: Control temporal y prevención superposición**
  - **Premisa:** Sistema no permite agendar al pasado ni tener doble agendamiento del mismo técnico en un bloque.
  - **Validación testing:** El caso INV-01 (Fecha en el pasado) y el caso TENSION-05 (Bloque horario de hoy que ya transcurrió) validan que el sistema intercepta correctamente estos errores a nivel de modelo antes de tocar la base de datos. El caso INV-02 y BORDE-04 comprueban que el sistema rechaza exitosamente asignar a un técnico a un bloque ocupado, pero sí lo permite si la reserva original cambia su estado a "CANCELADA", liberando el bloque.

12. Uso de IA en el proyecto

Como herramientas de IA se utilizó Copilot, Antigravity, con Gemini 3 Pro como LLM, y se utilizó para generar una versión inicial de la aplicación web utilizando la definición mejorada de los requerimientos. Luego de ejecutar el código generado, y hacer pruebas sobre dicha versión, se corrigieron manually los errores detectados.

Se delegó solo el desarrollo a la IA pensando en que actualmente, el rol de desarrollador de software está cambiando, y se está volviendo esencial precisamente ser capaz de definir los requerimientos de manera precisa, y ser capaz de evaluar críticamente el código generado por la IA.

A la IA no se le delegó el diseño de la estrategia de pruebas, ni la ejecución de pruebas, ni el análisis de los resultados de las pruebas. Tampoco se le delegó el análisis de los supuestos, ni el análisis y mejora de los requerimientos. Nada de esto porque siguiendo a lo dicho en el párrafo anterior, el valor del desarrollador hoy en día radica en ser capaz de definir todos estos puntos.

13. Mejoras y trabajo futuro

Debido a que el equipo pasó de 2 a 3 integrantes, se realizó un cruce de ideas y percepciones respecto de cómo definir y analizar los nuevos requerimientos, de forma tal, que está sección nace debido a que por temas de tiempo y para entregar un trabajo correcto, no se incluyeron cierta ideas, mejoras o interpretaciones que nacieron al cruzar las ideas de los integrantes del equipo. Así mismo, se mapean y establecen las mejoras y como debería mejorar el sistema en futuras iteraciones, tal que la solución sea mucho más robusta.

A continuación, se definen los siguientes requerimientos mejorados para su posterior implementación.

- Clientes pueden ser creados, listados, eliminados, y editados.
- Para crear un cliente se requiere añadir la siguiente información a complementar:
    - Apellido, String de máximo 50 caracteres.
    - RUT, formato chileno, sin puntos ni guión. (NO MAS ID, SOLO RUT)
    - Email, formato email válido.
- Se pueden eliminar clientes, solo sí el cliente canceló todas sus reservas o no tiene reservas pendientes. No se pueden eliminar clientes que tengan reservas ya agendadas y pendientes.
- Si el estado del técnico pasa a inactivo teniendo una reserva a futuro, lo ideal sería que se asignará un técnico al cliente que solicitó la reserva.
- Técnicos pueden ser eliminados y editados.
- Las categorías soportadas por un técnico deben estar previamente definidas en el sistema. Si un técnico posee una categoría que no está en el sistema, se debe crear antes de agregar a técnico.
- Para crear un técnico se requiere añadir la siguiente información:
    - CAMBIAR ID POR RUT.

    - Apellido, String de máximo 50 caracteres.
    - RUT, formato chileno, sin puntos ni guión.
    - Email, formato email válido.
    - Teléfono, mínimo 8 dígitos, máximo 12 dígitos. Sin código país.
    - Dirección, String de máximo 255 caracteres.
    - Categorías (1 o más) de electrodomésticos que puede reparar. Elegibles desde un listado de categorías existentes.
- Al técnico por supuestos operativos, por ejemplo, transporte entre reservas, no puede tener otra reserva en el bloque siguiente, el técnico solo puede operar bloque por medio.
- Se pueden eliminar(inactivar) técnicos, solo sí no tiene reservas pendientes, pero no se pueden eliminar técnicos que tengan reservas ya agendadas y pendientes.
- Categorías de electrodomésticos pueden ser creadas, listadas, eliminadas, y editadas.
- Para crear una categoría se requiere la siguiente información:
    - Nombre
    - Descripción
- Se pueden editar categorías, pero no se pueden editar categorías que ya estén asociadas a reservas ya agendadas y pendientes.
- Se pueden eliminar categorías, pero no se pueden eliminar categorías que ya estén asociadas a reservas ya agendadas y pendientes. Solo se puede eliminar categorías que no estén asignadas a un técnico.
- Reservas pueden ser creadas, listadas, pospuestas a futuro, y canceladas. No deben ser eliminables para tener registro histórico.
- Categoría de electrodoméstico a reparar, seleccionada desde listado de categorías. El listado debe mostrar sólo categorías que el técnico pueda reparar.
- Estado de la reserva, puede ser "pendiente", "completada", "cancelada", "expirada". El valor por defecto es "pendiente".
- Al listar reservas, si el técnico indica que llegó a la visita, su estado cambia a "completada" . De lo contrario si excede la fecha de la reserva y no ha sido “completada” esta pasa a estado “expirada”.
- Las reservas deben ser filtrables por estado, técnico, cliente, y categoría. Ordenables por fecha y hora, por orden alfabético de cliente, o por orden alfabético de técnico

14. Reflexión final

El error o problema más importante detectado durante el desarrollo, cómo fue detectado; Un punto que no se consideró inicialmente fue la categoría desde el enfoque técnico, ya que es clave para asegurar que los técnicos sean asignados correctamente según sus capacidades. Incorporarlo mejora la coherencia del sistema y evita errores en la asignación de reservas, así aseguramos que cada técnico pueda hacer su trabajo de la mejor manera.

No se detectó antes debido a que los requisitos estaban muy acotados y, al tratarse de una empresa pequeña, este nivel de detalle no parecía necesario en una primera instancia. Ya que se asumió o se suponía, que el administrador del sistema tenía conocimiento sobre los técnicos asignables , ya que estos son acotados, debido a que se trata de una empresa pequeña.

En general trabajamos bien (Discord, GitHub, WhatsApp) y no identificamos problemas relevantes en la organización. Lo único distinto fue la incorporación de un tercer integrante muy cerca de la entrega, lo que implicó adaptarse rápidamente, pero se logró integrar sin afectar significativamente el resultado. Una limitación actual es que las direcciones se ingresan manualmente, lo que puede generar errores. Como mejora futura, se propone integrar la Google Maps Places Autocomplete API, permitiendo autocompletar y validar direcciones, evitando así errores y posibles asignaciones a ubicaciones incorrectas.

15. Conclusiones

En este trabajo se logró transformar un requerimiento inicial incompleto en una solución más clara y estructurada, definiendo reglas de negocio concretas que permitieron implementar un sistema funcional.

Se evidenció la importancia de la validación y verificación en el desarrollo de software, ya que permitió asegurar tanto que el sistema cumple con lo definido, como que responde a la necesidad real del negocio .

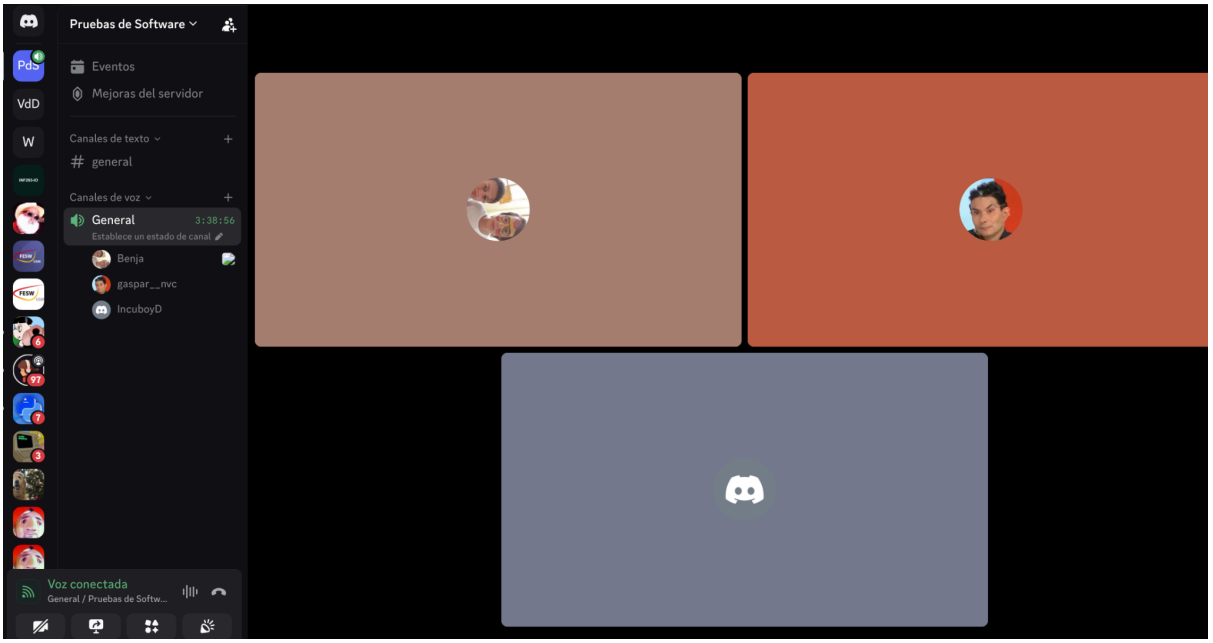
Además, el uso de testing fue clave para detectar errores y validar los supuestos planteados, reforzando la relación entre requisitos y casos de prueba. Finalmente, el proyecto permitió entender que una buena definición de requerimientos es fundamental para evitar problemas en etapas posteriores del desarrollo.

16. Evidencias del trabajo

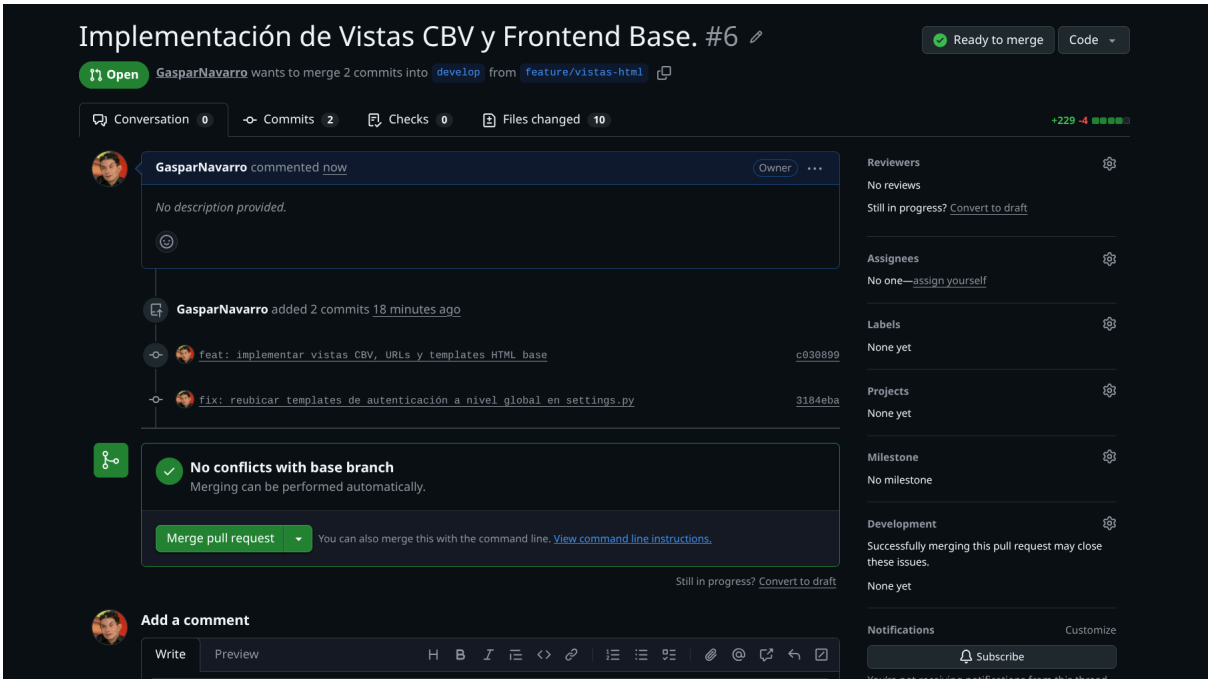
- Servidor de la aplicación en ejecución

```
^C(venv) gaspa@penguin:~/Tarea-1-Pruebas-De-Software$ python manage.py runserver
WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/
[28/Mar/2026 23:29:56] "GET / HTTP/1.1" 302 0
Not Found: /accounts/login/
[28/Mar/2026 23:29:56] "GET /accounts/login/?next=/ HTTP/1.1" 404 3024
Not Found: /favicon.ico
[28/Mar/2026 23:29:57] "GET /favicon.ico HTTP/1.1" 404 3005
```

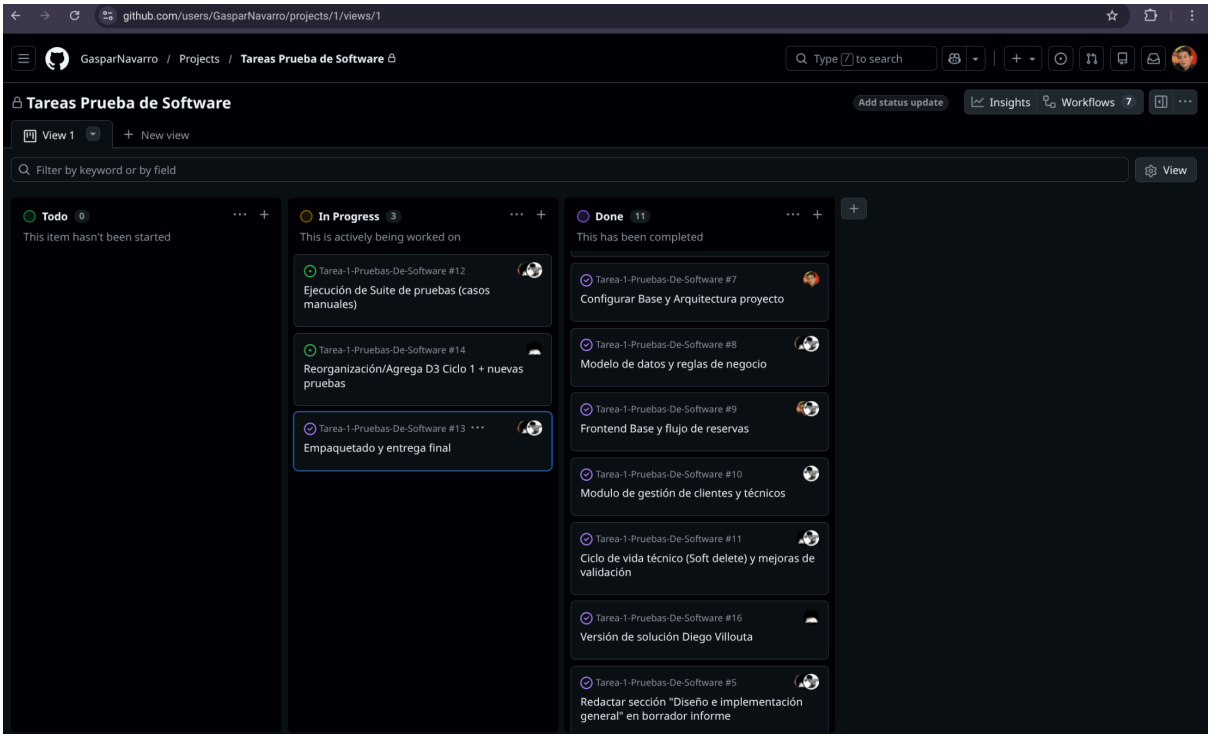
- Reunión de grupo en Discord



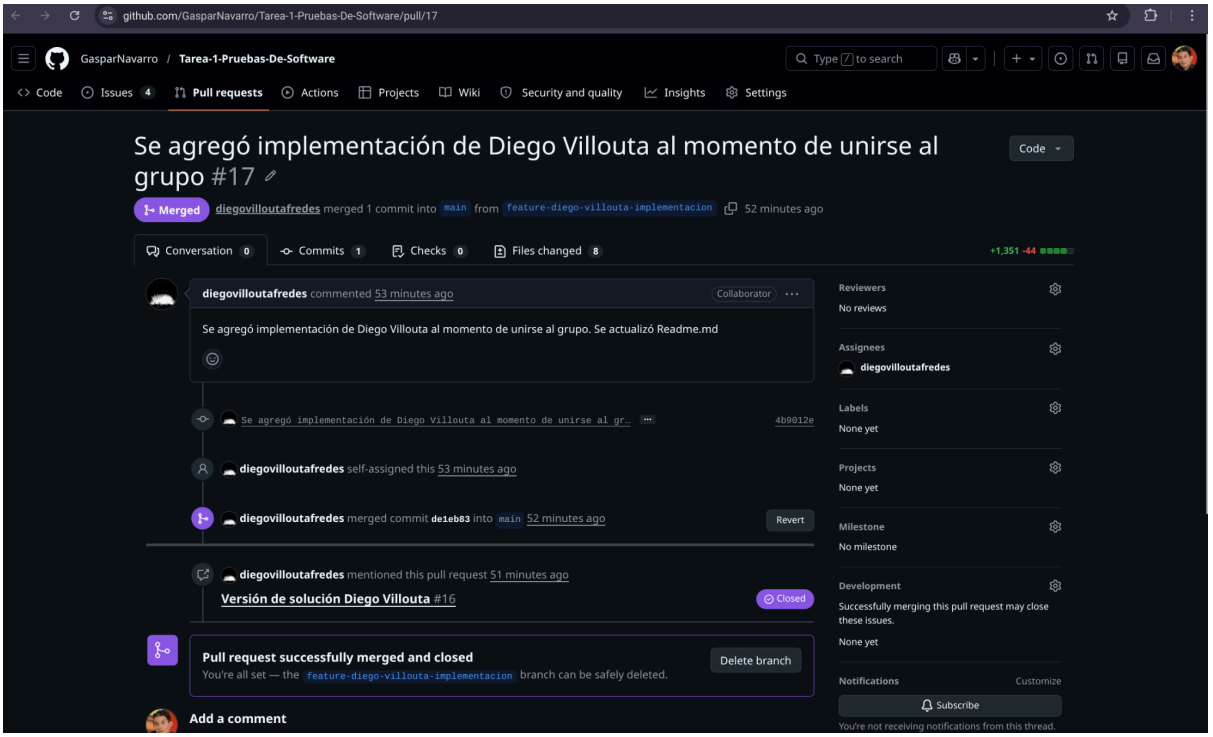
- Pull request de branch en GitHub



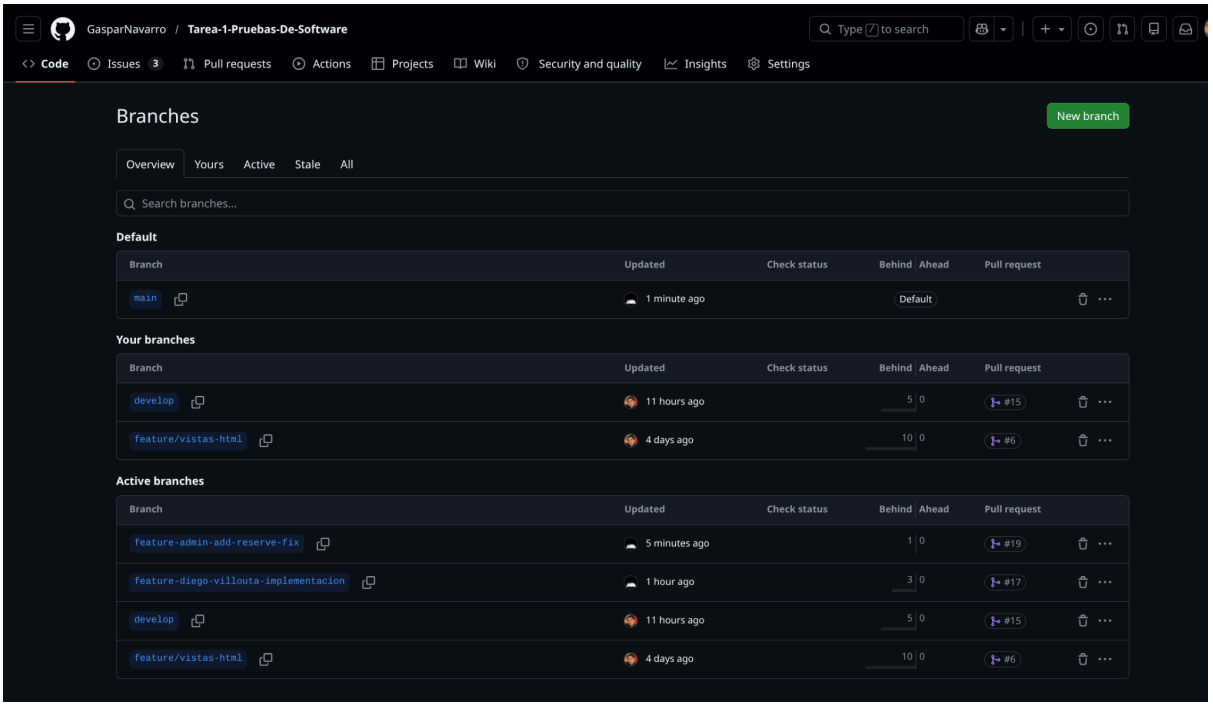
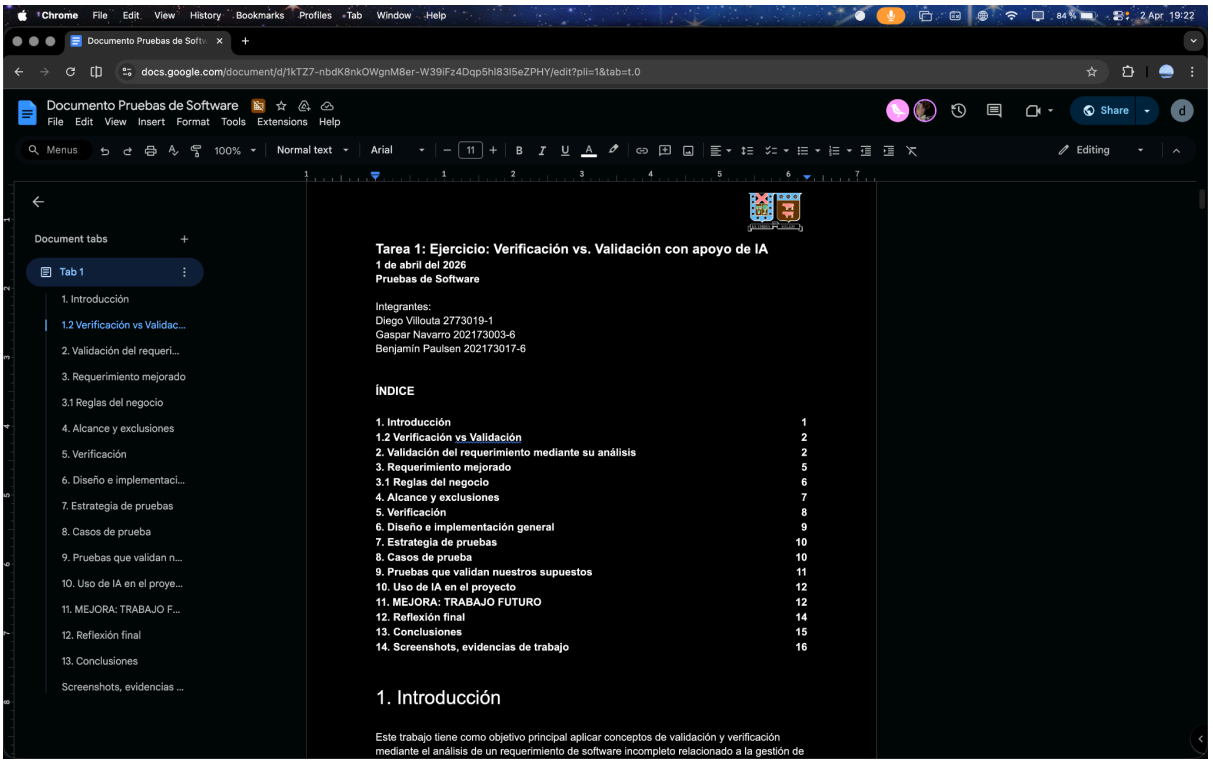
- Tablero de tareas en GitHub



- Pull request en repositorio en GitHub



- Branches de repositorio en GitHub.



- Elaboración del documento en conjunto, usando Google Docs.